Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	01 March 2026
Team ID	699d5f7bfdf409fdbd1bebf5
Project Name	ResearchHub AI: Intelligent Research Paper Management and Analysis System using Agentic AI
Maximum Marks	4 Marks

Technical Architecture:
The architecture includes:

- Application logic blocks
- Local & Cloud demarcation
- External APIs
- Data storage components
- Machine Learning model interfaces

```
[User Browser]
↓
[React + TypeScript Frontend]
↓
[FastAPI Backend Server]
↓
-----------------------------------------------
|      Core Application Layer                  |
| - Authentication Module (JWT)                |
| - Workspace Management                       |
| - Paper Search Engine                        |
| - PDF Parsing Engine                         |
| - RAG Pipeline                               |
-----------------------------------------------
↓
-----------------------------------------------
| AI & Intelligence Layer                      |
| - Embedding Model                            |
```

```
| - Vector Database (pgvector)          |
| - Groq Llama 3.3 70B API              |
-----------------------------------------------
                    ↓
-----------------------------------------------
| Data Layer                            |
| - PostgreSQL Database                 |
| - Object Storage (PDFs)               |
-----------------------------------------------
                    ↓
              External APIs:
              - ArXiv API
              - Semantic Scholar API
              - Groq API
```

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1	User Interface	Web-based interface for researchers to search, manage papers, and interact with AI chatbot	React.js, TypeScript, Tailwind CSS
2	Application Logic-1	Backend API services (auth, workspace, search, paper management)	Python, FastAPI
3	Application Logic-2	PDF parsing and text extraction engine	PyMuPDF / PDFMiner
4	Application Logic-3	RAG pipeline & contextual query processing	Python, LangChain (optional), Custom RAG logic
5	Database	Structured data storage (users, workspaces, metadata, chats)	PostgreSQL
6	Cloud Database	Managed PostgreSQL with vector extension	PostgreSQL + pgvector
7	File Storage	PDF storage and retrieval	AWS S3 / Cloud Object Storage
8	External API-1	Research paper search & metadata	ArXiv API
9	External API-2	Citation & paper metadata enrichment	Semantic Scholar API
10	Machine Learning Model	Context-aware question answering & synthesis	Groq Llama 3.3 70B
11	Infrastructure (Server / Cloud)	Backend deployment & hosting	AWS EC2 / Render / Docker

Infrastructure Demarcation

Local Development

- Docker Compose
- Local PostgreSQL

- Local FastAPI server
- Local React dev server

Cloud Deployment

- Frontend → Vercel
- Backend → AWS EC2 / Render
- Database → Managed PostgreSQL
- Object Storage → AWS S3
- Monitoring → CloudWatch / Logs

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Backend and frontend built on open-source frameworks	React.js, FastAPI, PostgreSQL
2	Security Implementations	Secure authentication, encryption, role-based access	JWT, HTTPS, bcrypt hashing, OWASP best practices
3	Scalable Architecture	3-tier architecture with modular AI service layer	Microservices-ready, Dockerized
4	Availability	Cloud-hosted backend with managed DB and load balancing	AWS EC2, Managed DB, Nginx
5	Performance	Optimized vector search, async APIs, caching	pgvector indexing, Redis (optional), Async FastAPI

Security Architecture

- JWT-based authentication
- Password hashing (bcrypt)
- Role-based workspace isolation
- CORS configuration
- API rate limiting
- HTTPS encryption
- Secure environment variable management

Performance Design Considerations

- Asynchronous FastAPI endpoints
- Vector indexing using pgvector
- Chunk-based embedding storage
- Streaming responses for AI chat
- Pagination for search results
- Query optimization for metadata

Expected capability:

- 1000+ concurrent users
- < 2 sec metadata search latency
- < 4 sec AI response latency

Machine Learning Integration Layer
Model Interface

- User Query → Embedding Generation
- Similarity Search → Top-K chunk retrieval
- Context Injection → Groq LLM
- Response returned with grounding

AI Features

- Context-aware Q&A
- Multi-paper synthesis
- Cross-paper comparison
- Citation-grounded outputs

 Architectural Style

- 3-Tier Architecture
    - Presentation Layer (React)
    - Application Layer (FastAPI)
    - Data Layer (PostgreSQL + Storage)
- AI Service Layer (RAG Engine)
- Modular & Microservice-ready design